```
  model: google.textEmbeddingModel("text-embedding-004"),
  value: "The quick brown fox jumps over the lazy dog",
});

console.log(embedding);
```

Highlights the specification of the embedding model being used from the Google Gemini API

Points out the input text for which the embedding is generated, demonstrating how to provide a specific value for analysis

The `model` parameter specifies the embedding model to use, and the `value` is the input text for which you want to generate an embedding.

The purpose of `embedMany` is to generate embeddings for multiple independent input values in a single call. This is particularly efficient when you have a batch of text data that you want to process simultaneously.

Listing 5.8 Creating multiple embeddings

Indicates the import of the embedMany function, which is essential for generating embeddings from text values

```
import { embedMany } from "ai";
import { createGoogleGenerativeAI } from "@ai-sdk/google";

const google = createGoogleGenerativeAI({ apiKey });

const { embeddings } = await embedMany({
    model:
google.textEmbeddingModel("text-embedding-004"),
    values: [
        'The quick brown fox jumps over the lazy dog',
        'A journey of a thousand miles begins with a single step',
        'To be or not to be, that is the question',
    ],
});

console.log(embeddings);
```

Highlights the specification of the embedding model being used from the Google Gemini API

Points out the input values for which the embedding is generated, demonstrating how to provide multiple values for analysis

The values parameter is not a single text but a list of text strings. The model will return the list of embeddings that correspond to each provided input string in the same order.

If you run any of the examples provided, you will get a bunch of floating-point numbers back:

```
$ node ch05/embeddings.google.js
[
     -0.0450604, -0.014942412,
   -0.037350334, -0.025503622,
    0.033759177,  0.030023085,
   0.0025531165, -0.016676709,
  -0.0071603837, -0.011624326
]
```

What can you really do with those numbers? One of the most effective ways to utilize embeddings is by storing them in a database and employing them for similarity